

```
# Define Smart City Infrastructure
class SmartCity:
    def __init__(self, grid_id):
        self.grid_id = grid_id
        self.energy_source = 'WIND_TURBINES'
        self.connectivity_protocol = 'IoT_MESH_V5'
        self.urban_layout = 'MODULAR_GRID'
```

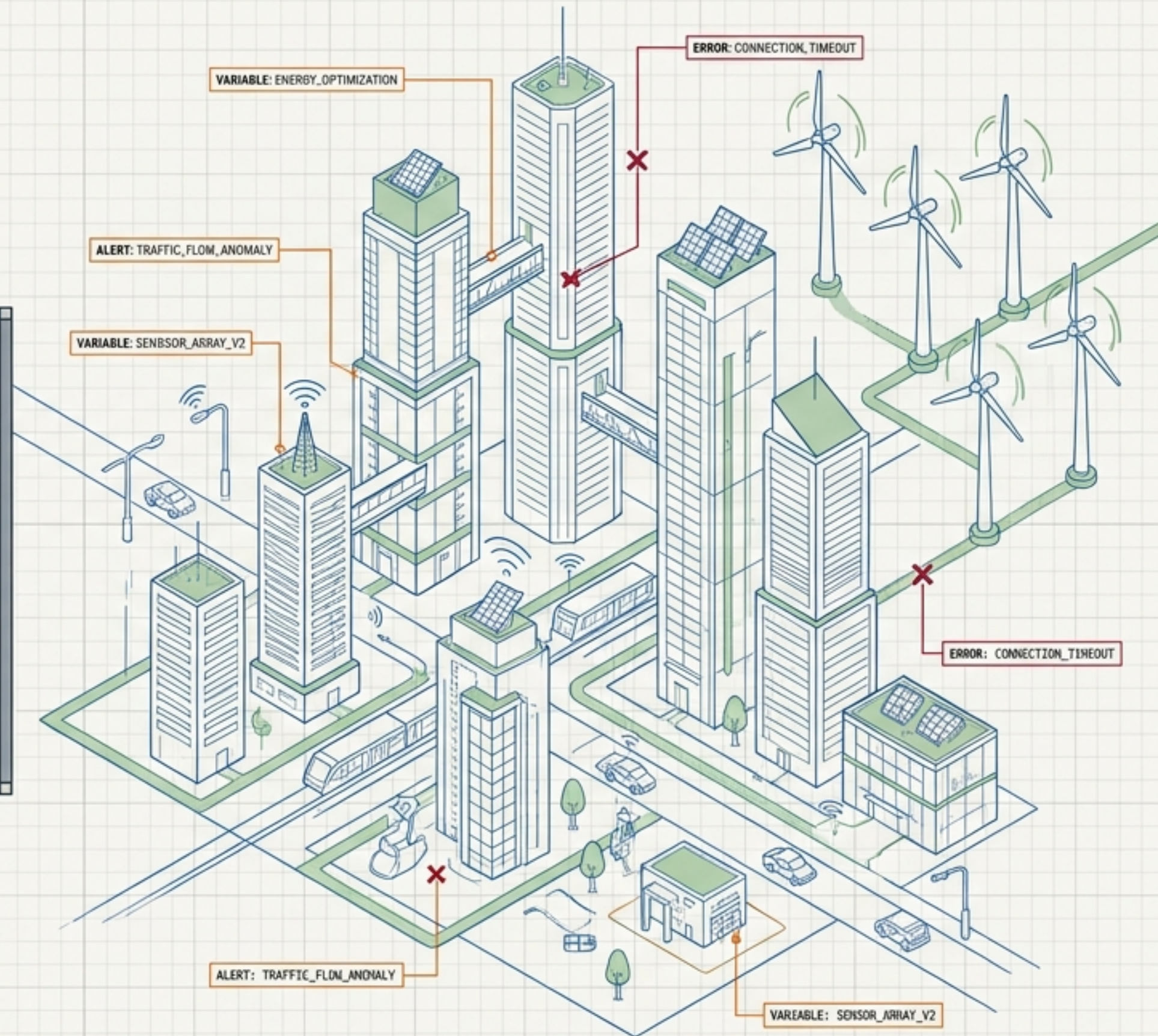
```
def construct_buildings(self, blueprints):
    # Process blueprints for urban structures
```

De la Sintaxis a la Realidad: Cómo el Código Construye el Mundo Conectado

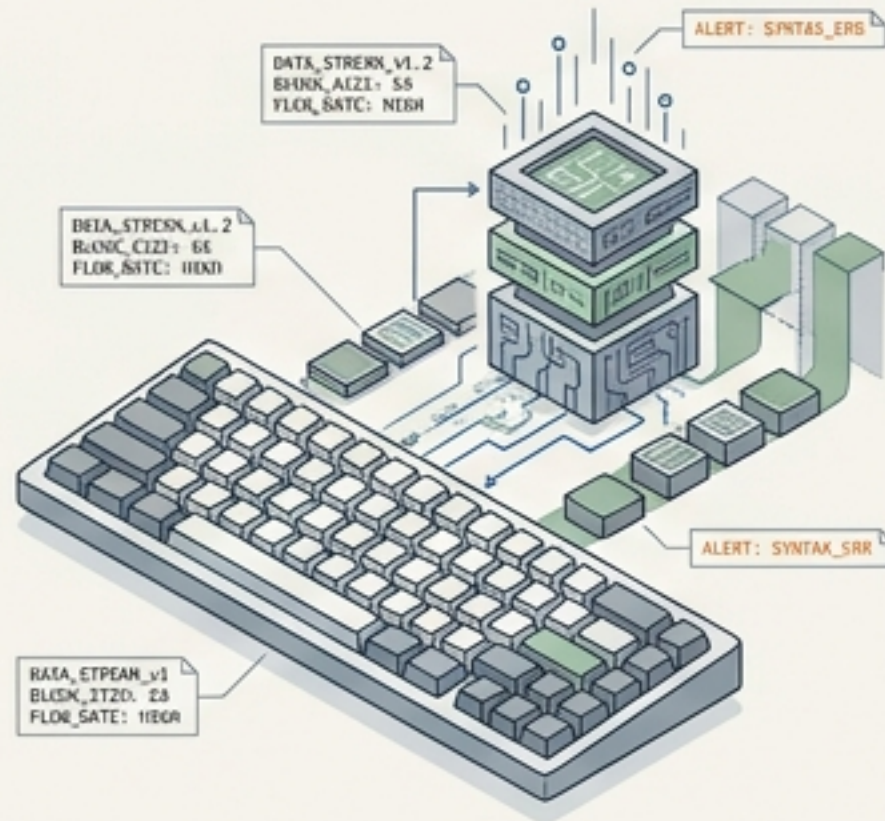
Una guía visual sobre lenguajes de programación, modularización y la arquitectura del Internet de las Cosas (IoT).

```
def construct_buildings(self, blueprints):
    # Process blueprints for urban structures
    for building in blueprints:
        self.deploy_structure(building.type, building.location)
        print(f'BUILDING_{building.id}: {building.type} DEPLOYED')
```

```
def activate_sensors(self):
    # Initialize environmental and traffic sensors
    print('SENSORS: ACTIVATED ON GRID')
```



La Programación Textual: El Estándar Profesional



Utiliza sintaxis y gramática precisas.
[SYNTAX: PRECISE]; [GRAMMAR: EXACT]

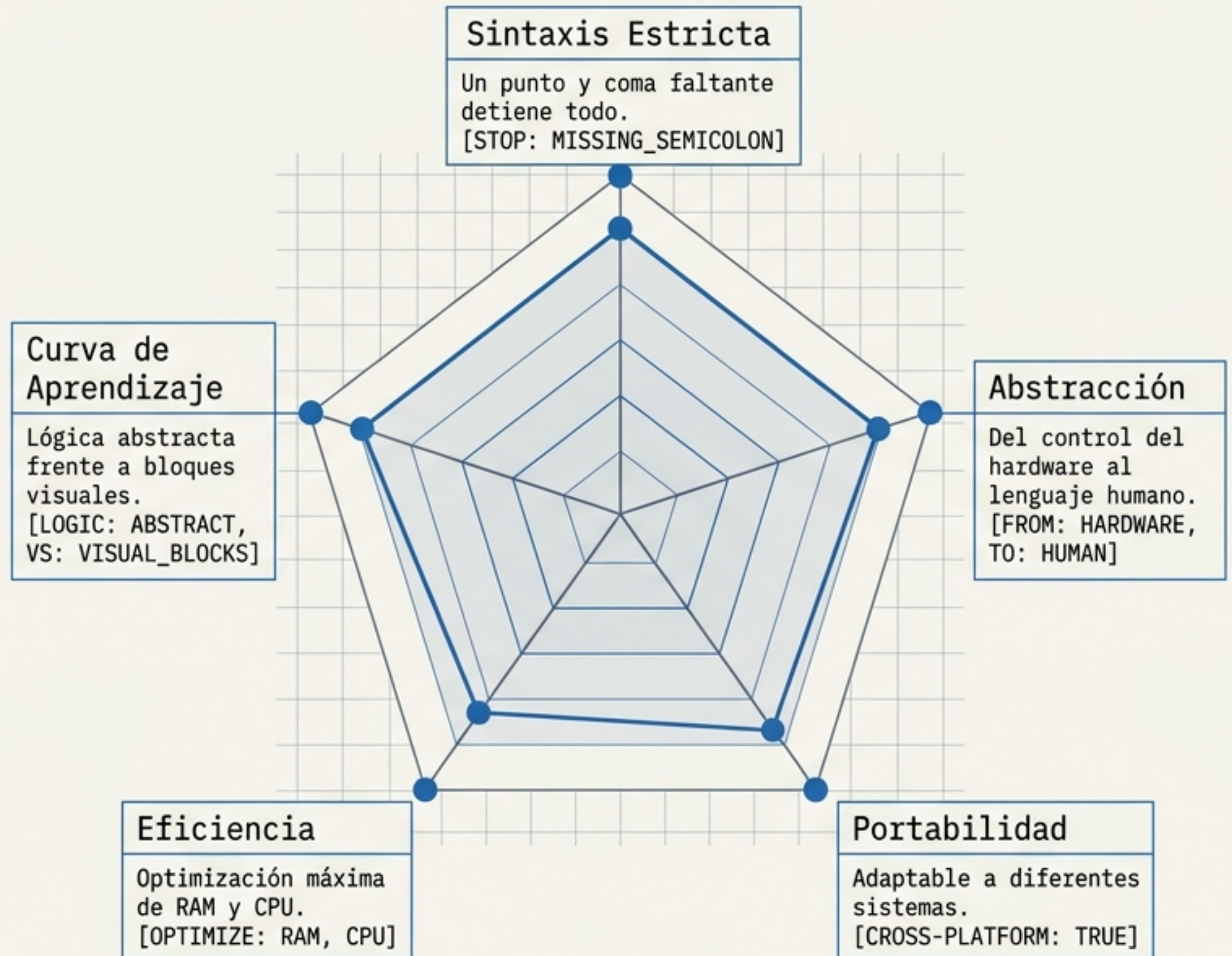


Alto nivel de abstracción.
LEVEL: HIGH_ABSTRACTION

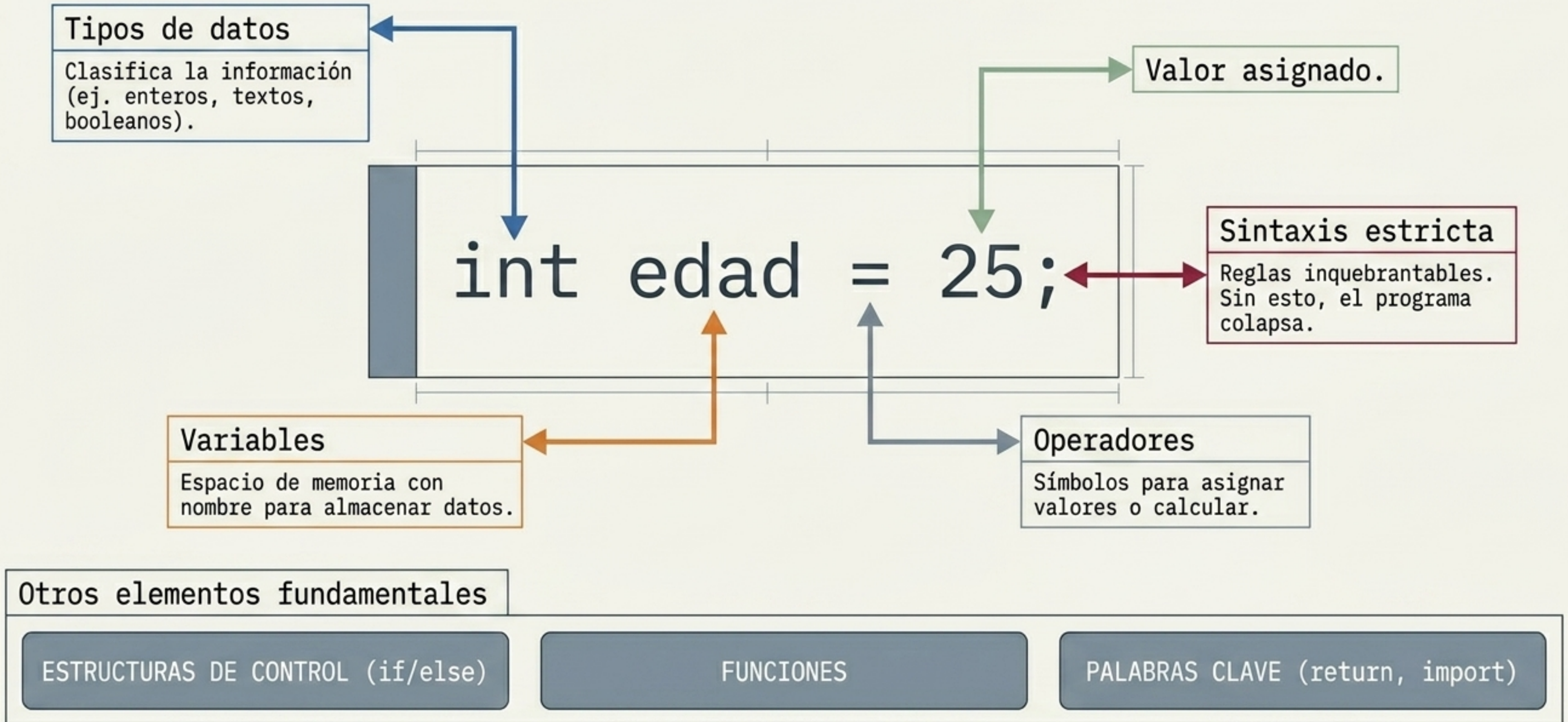


Requiere estructura y vocabulario exactos.

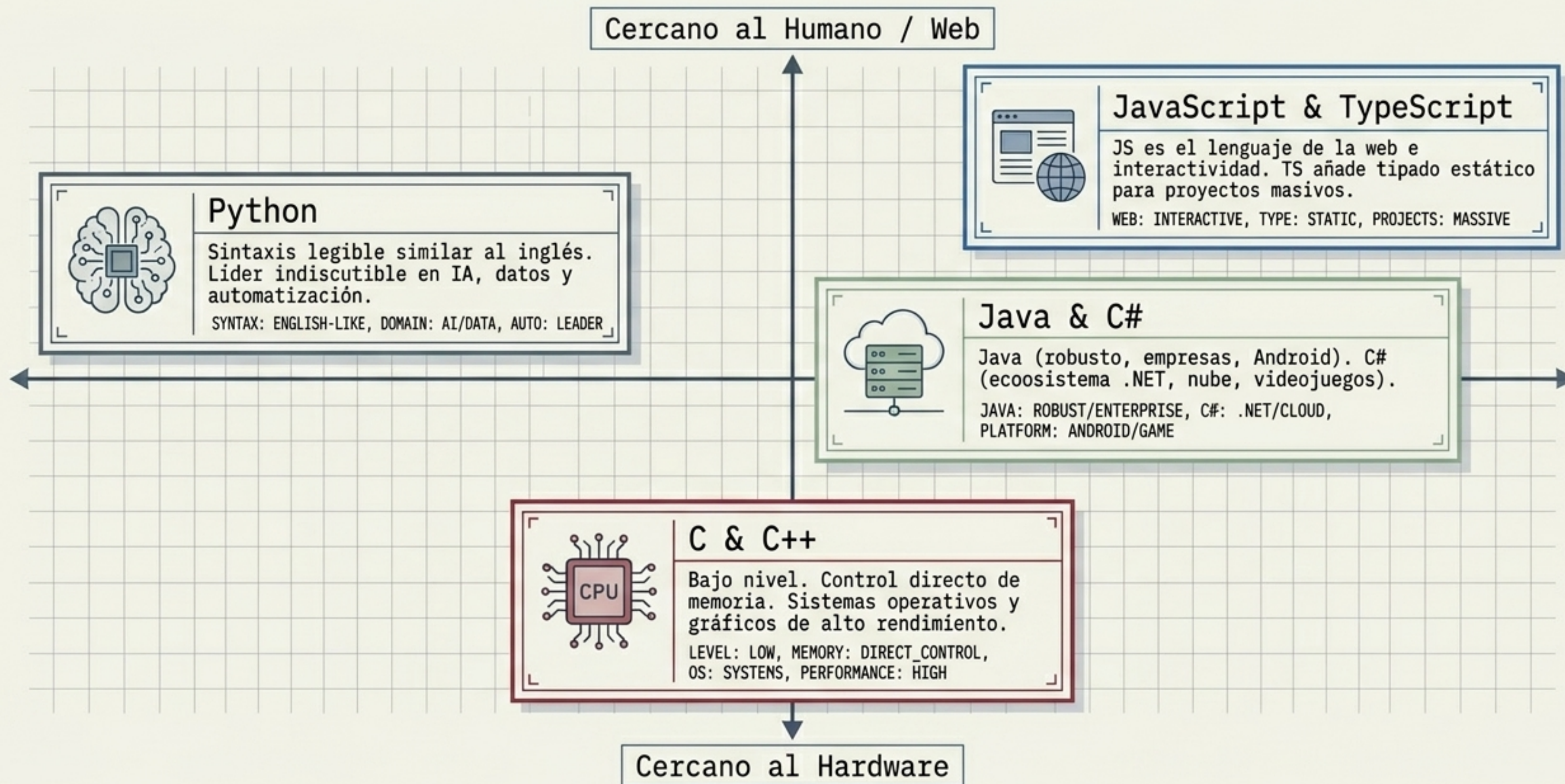
STRUCTURE: REQUIRED; VOCAB: DEFINED



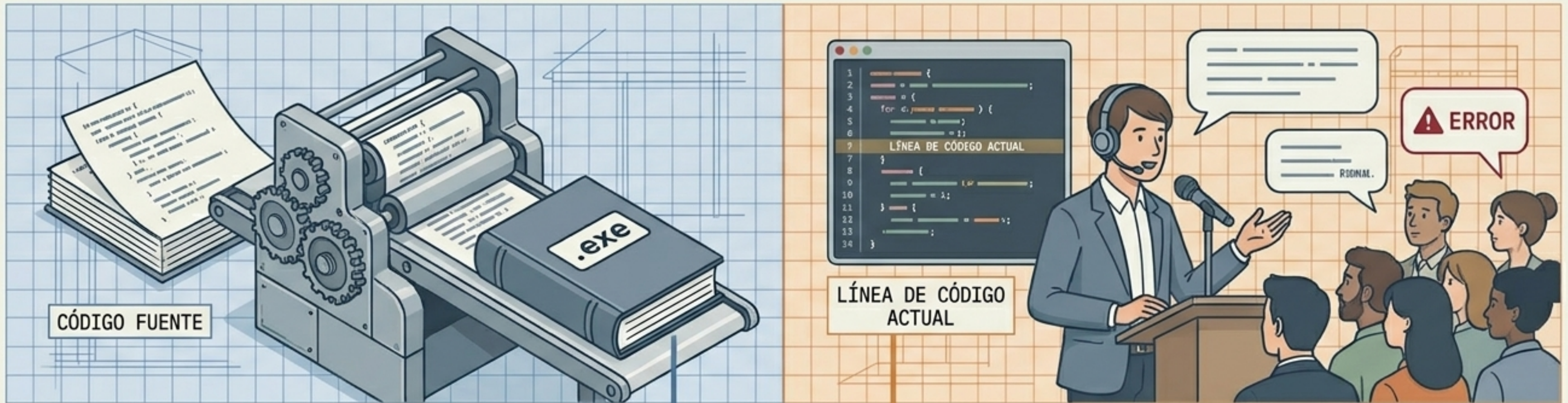
Anatomía del Código: Desglose de una Instrucción



El Ecosistema de los Lenguajes



La Traducción: Compilación vs. Interpretación



El Compilador: Traduce todo de golpe antes de ejecutar.

El Intérprete: Lee y ejecuta línea por línea en tiempo real.

Compilados (C, C++, Rust)

Traduce a código máquina binario.
Máxima velocidad.
Genera archivo ejecutable.

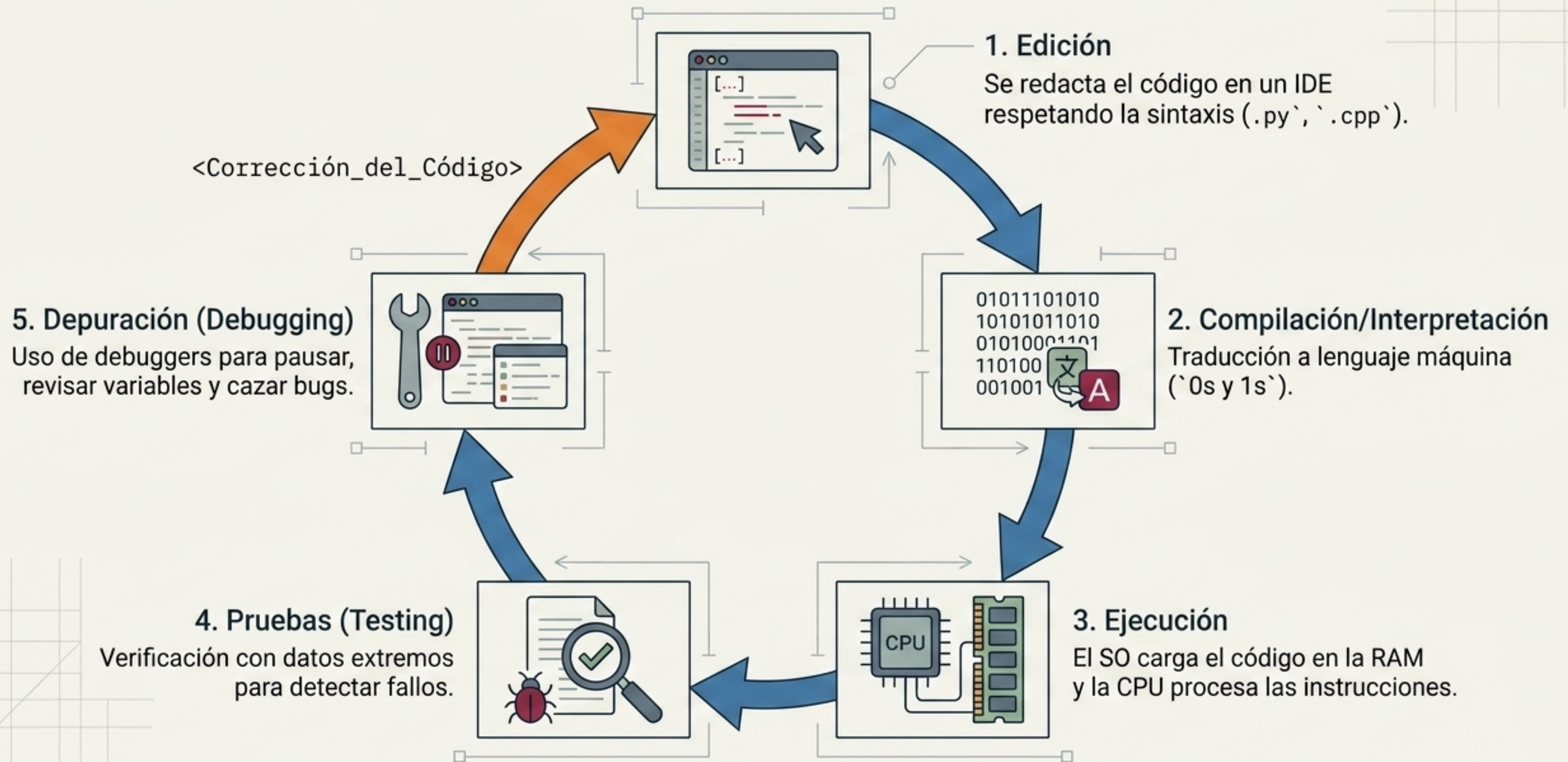
Interpretados (Python, JS, Ruby)

Intérprete actúa en tiempo real.
Máxima flexibilidad.
Se detiene en la línea del error.

Híbridos (Java, C#)

Compila a formato intermedio (bytecode) y se interpreta en una máquina virtual.

El Ciclo de Vida del Software Textual



Manejo de Errores: Interpretado vs. Compilado

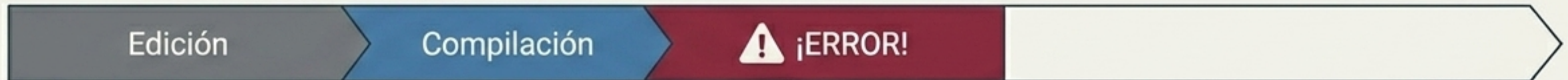
Escenario: Un programa simple con un error crítico (división entre cero).

Python (Interpretado)



No hay compilación previa. Arranca de inmediato y choca contra el error en tiempo real en la línea específica.

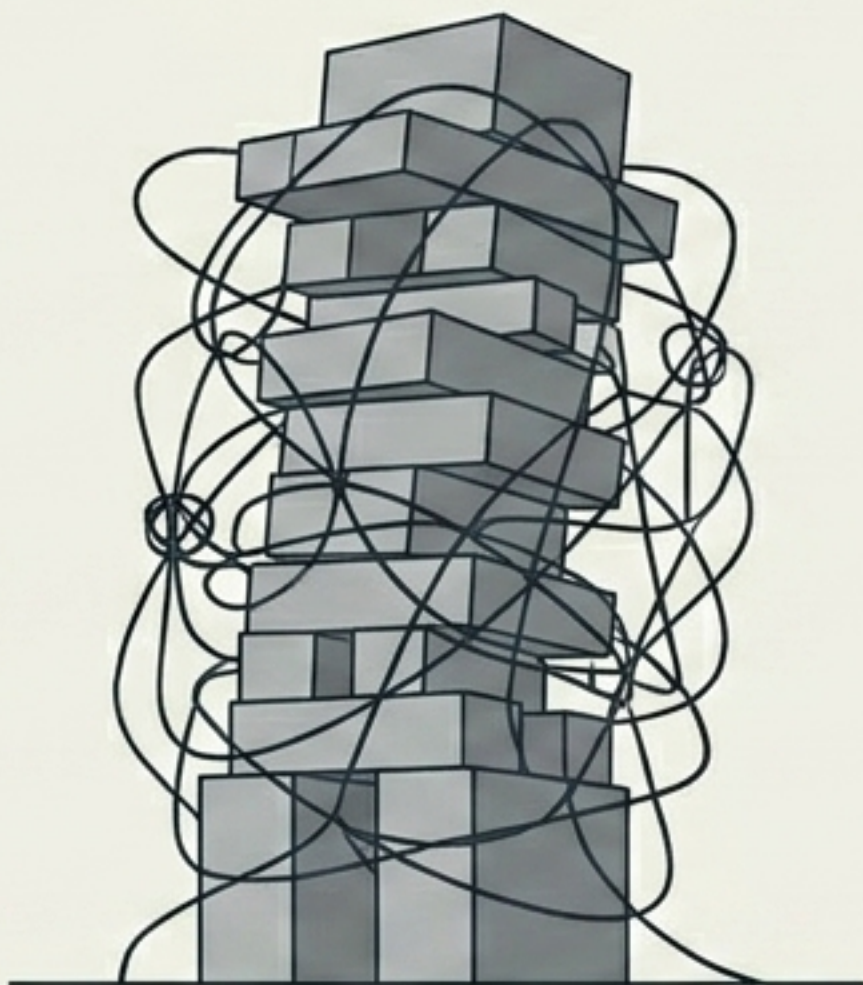
C++ (Compilado)



Analiza de arriba a abajo. Al detectar el error, aborta y NO genera el archivo ejecutable. La ejecución nunca ocurre.

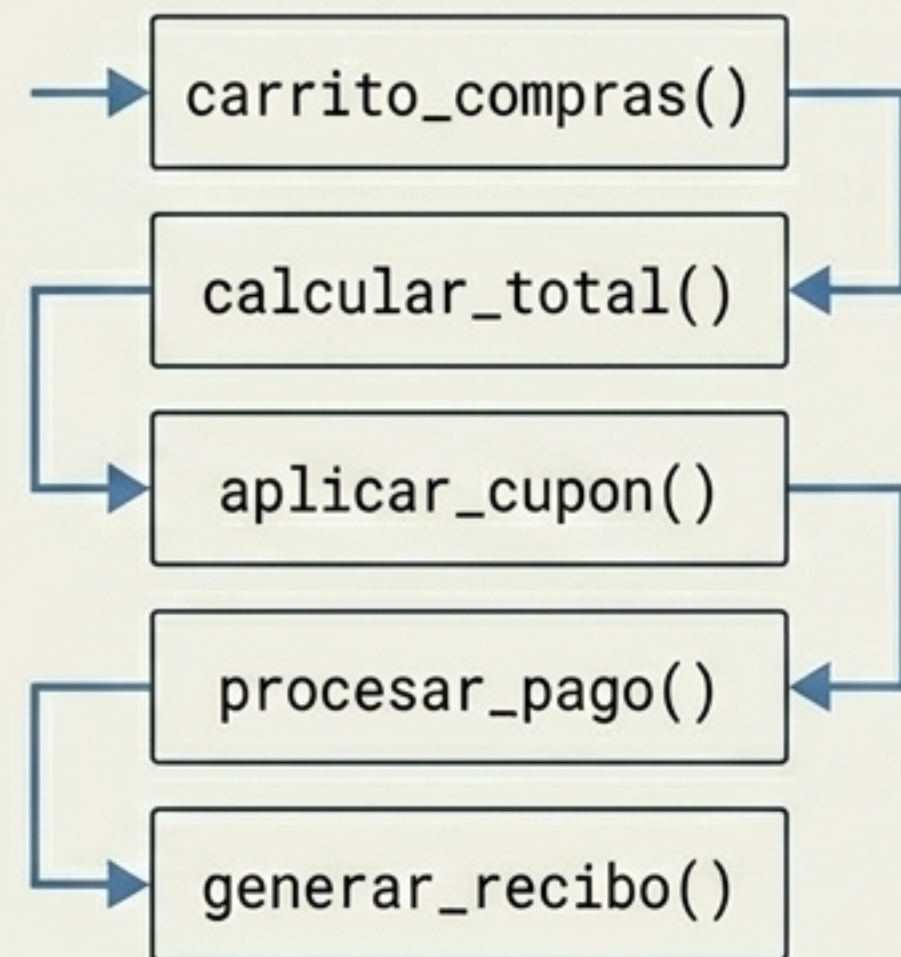
El Principio de Modularización: Divide y Vencerás

Enfoque Monolítico



El Código Espagueti.
500 líneas en un solo bloque.
Si falla el pago, colapsa toda la tienda.
Difícil de leer y mantener.

Enfoque Modular



- ✓ Reutilización.
- ✓ Facilidad de mantenimiento.
- ✓ Legibilidad de índice de libro.
- ✓ Trabajo en equipo simultáneo.

La Estructura Modular en la Práctica (Python)

modulos.py

```
def calcular_total(precio_base, impuesto):  
    return precio_base + (precio_base * impuesto)  
  
def aplicar_descuento(total, porcentaje):  
    return total - (total * porcentaje)
```

Bloques independientes y reutilizables.
Solo se modifican aquí si cambian las reglas del negocio.

main.py

```
precio = 100.0  
iva = 0.21  
total = calcular_total(precio, iva)  
precio_final = aplicar_descuento(total, 0.10)
```

El programa principal actúa como el director de orquesta.
Es limpio y fácil de leer.

Del Código a la Realidad Física: El IoT en Acción

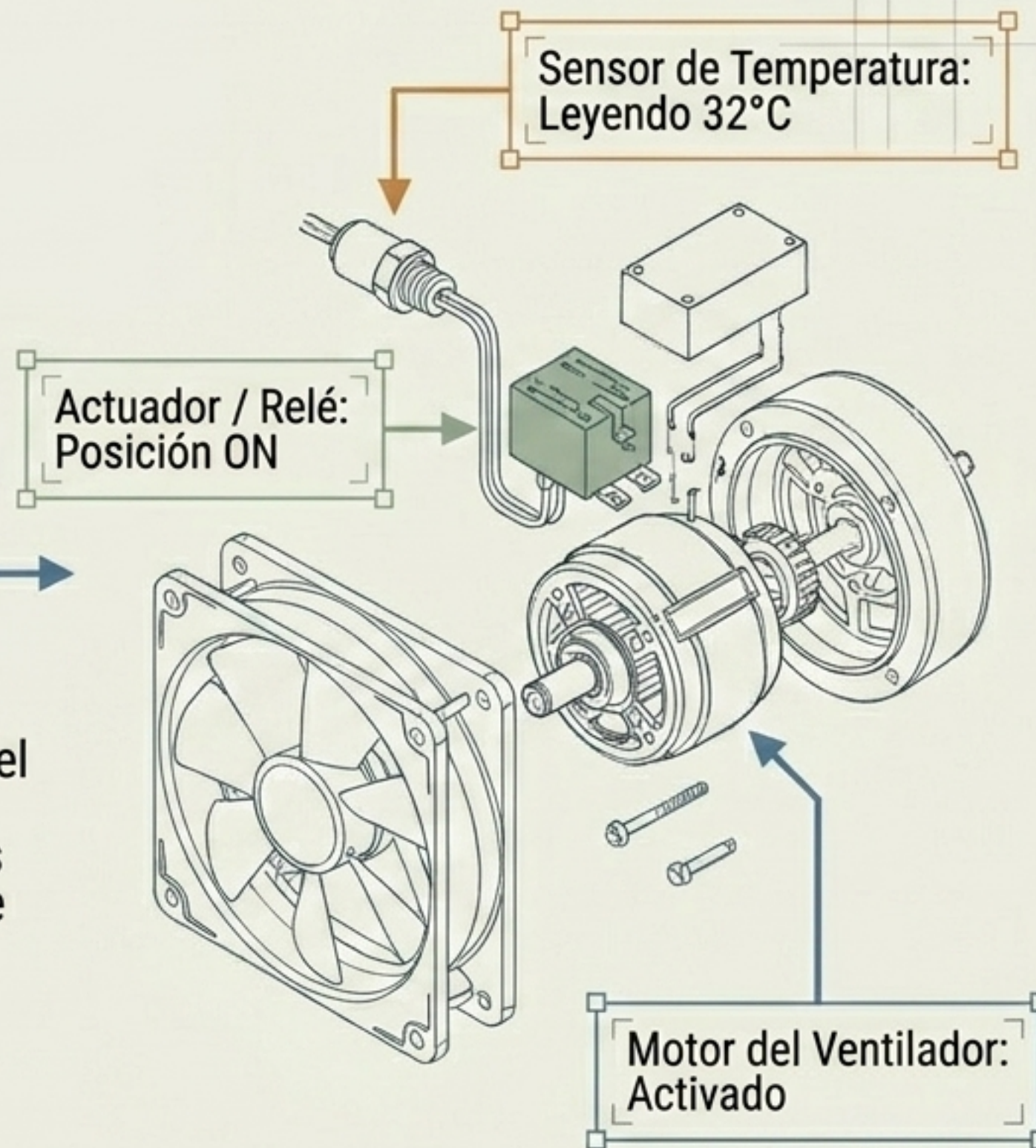
El Código

```
if temp_actual > 30.0:  
    activar_ventilador(True)
```

```
if temp_actual > 30.0:  
    activar_ventilador(True)
```

La Conexión

La Sintaxis Escapa del Ordenador. En el Internet de las Cosas (IoT), los módulos de código evalúan el mundo real y envían comandos físicos.



La Arquitectura de 4 Capas del IoT

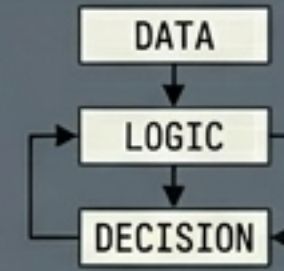
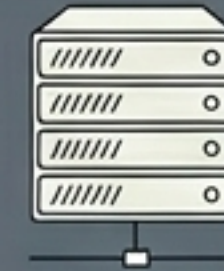
INTERFAZ (La pantalla)

Apps móviles o dashboards para control del usuario en tiempo real.



PROCESAMIENTO (El cerebro)

Servidores (locales o nube) que analizan datos mediante lenguajes textuales y toman decisiones lógicas.



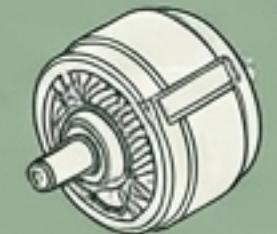
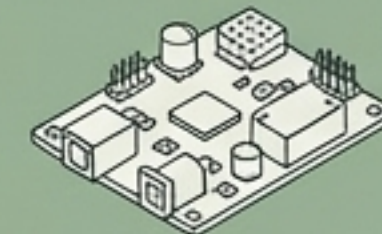
CONECTIVIDAD (El sistema nervioso)

Protocolos para enviar datos a la red (Wi-Fi, Bluetooth, Zigbee).



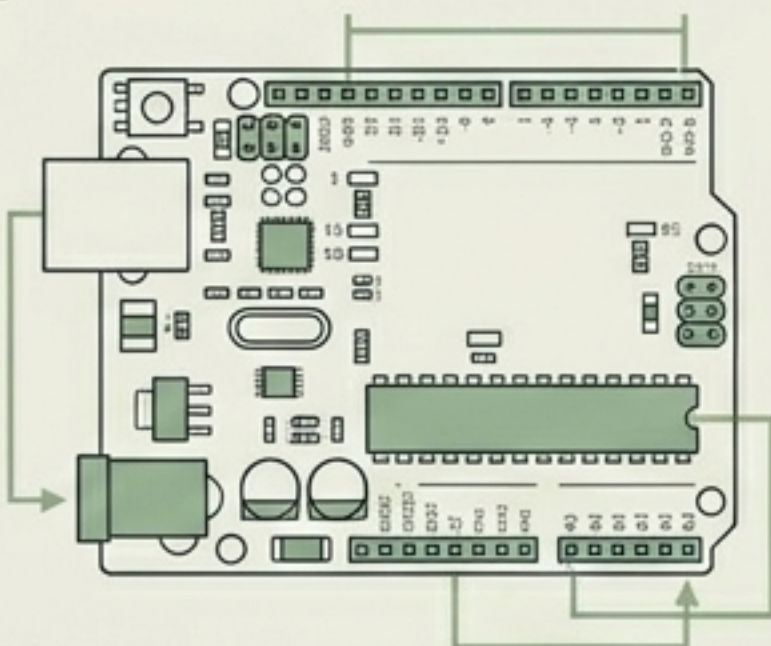
HARDWARE (Los ojos y las manos)

Sensores (temperatura, luz) y Actuadores (motores, relés).



Lenguajes Textuales en las Capas IoT

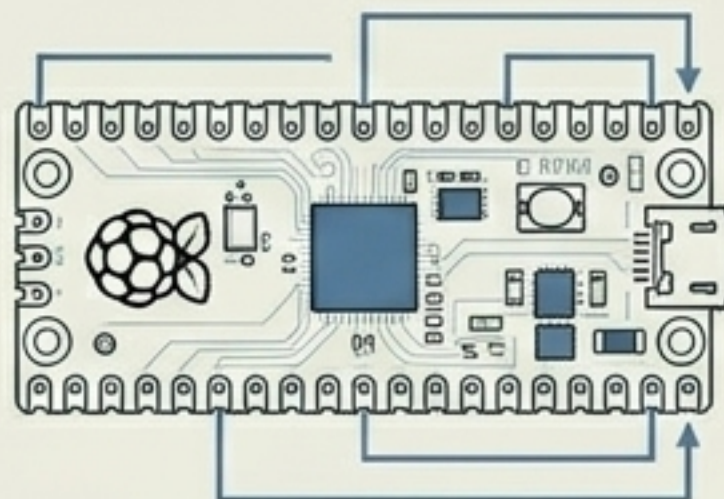
C y C++



Microcontroladores

Dominio absoluto por su gestión de memoria ultraeficiente en dispositivos con recursos y energía limitados.

MicroPython



Prototipado

Versión optimizada de Python para hardware. Ideales para desarrollo rápido por su sintaxis accesible.

Node.js (JS)

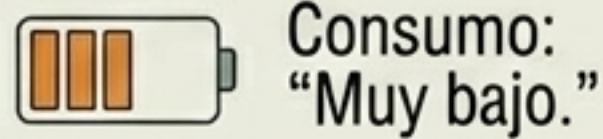
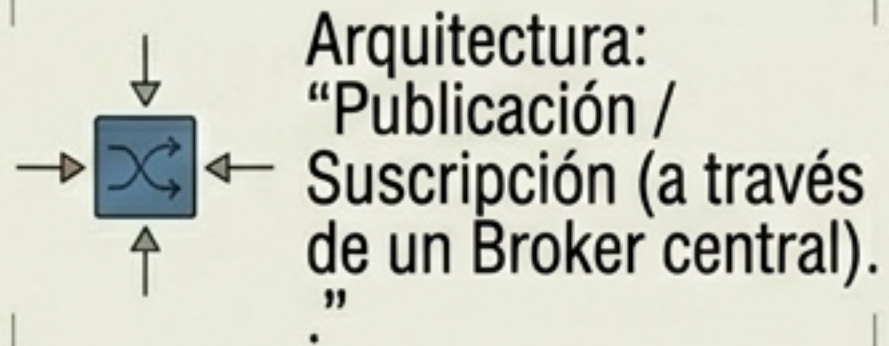


Servidores y Gateways

JavaScript domina el centro neurálgico, gestionando de forma asíncrona miles de conexiones simultáneas desde los sensores.

Capa de Aplicación: Protocolos de Mensajería IoT

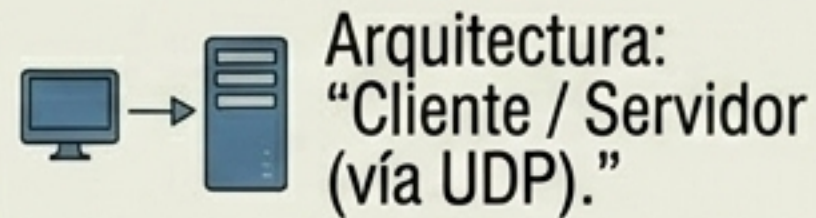
MQTT (El Estándar)



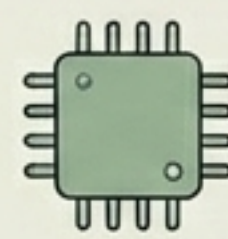
(Highlight with a small, )



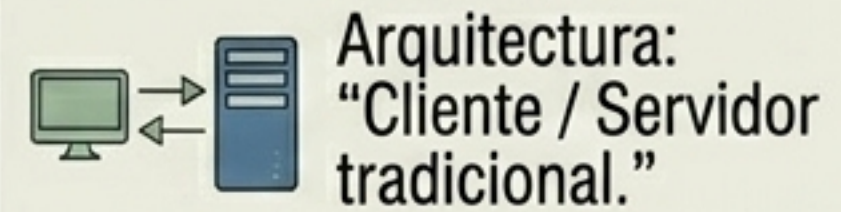
CoAP

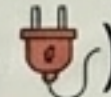


(Highlight with a max. )



HTTP / REST

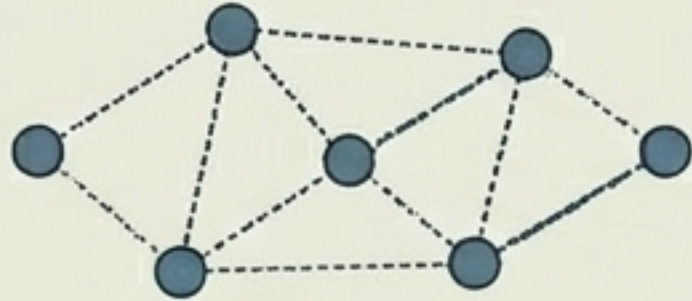


(Highlight with a plugged )



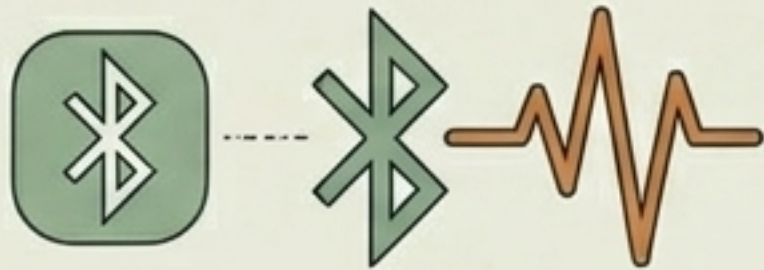
Capa de Red: El Espectro de Conectividad

Zigbee / Z-Wave



Redes en malla (Mesh). Cada dispositivo es un repetidor. Corto alcance (100m). Rey indiscutible de la domótica.

BLE - Bluetooth Low Energy



LoRaWAN



Largo alcance (hasta 15 km). Baterías que duran años. Paquetes pequeños de datos. Ideal para agricultura y ciudades.

BLE - Bluetooth Low Energy

Consumo bajísimo, corto alcance. Utilizado masivamente para dispositivos médicos, pulseras y beacons en interiores.

NB-IoT / LTE-M



NB-IoT / LTE-M

Usa redes móviles (4G/5G). Cobertura global y alta penetración en sótanos. Ideal para logística internacional y flotas.

IoT en Acción: Desde el Sensor hasta el Actuador

Domótica (Hogar)



Problema: Desperdicio energético.

Sensor de presencia

Evalúa tiempo vacío
(Código)

Apaga luces/AC (Actuador)

Agricultura de Precisión



Problema: Gasto excesivo de agua.

Sensor de humedad (<30%)

Lógica modular

Abre electroválvula de agua
(Actuador)

Smart Cities



Problema: Rutas de basura ineficientes.

Sensores de ultrasonido
(LoRaWAN)

Algoritmo en nube calcula
ruta (Python)

Optimización de camiones

Conclusión: El código ya no solo vive en pantallas; moldea y optimiza nuestro entorno físico.